



Java LinkedHashSet

[< Previous](#)[Next >](#)

Java LinkedHashSet

A **LinkedHashSet** is a collection that stores **unique elements** and **remembers the order they were added**.

It is part of the **java.util** package and implements the **Set** interface.

Tip: Use **LinkedHashSet** when you want a set that does not allow duplicates *and* keeps the original insertion order.

Create a LinkedHashSet

Example

Create a **LinkedHashSet** object called **cars** that will store strings:

```
import java.util.LinkedHashSet; // Import the LinkedHashSet class
```



Now you can use methods like `add()`, `contains()`, and `remove()` to manage your collection.

Add Elements

To add elements to a `LinkedHashSet`, use the `add()` method:

Example

```
import java.util.LinkedHashSet;

public class Main {
    public static void main(String[] args) {
        LinkedHashSet<String> cars = new LinkedHashSet<>();
        cars.add("Volvo");
        cars.add("BMW");
        cars.add("Ford");
        cars.add("BMW"); // Duplicate
        cars.add("Mazda");

        System.out.println(cars);
    }
}
```

Try it Yourself »

Output: The elements will appear in the order they were added (e.g., [Volvo, BMW, Ford, Mazda]).



Check if an Element Exists

Use `contains()` to check for an element:

Example

```
cars.contains("Mazda");
```

[Try it Yourself »](#)

Remove an Element

Use `remove()` to remove an element:

Example

```
cars.remove("Volvo");
```

[Try it Yourself »](#)

Remove All Elements

Use `clear()` to remove all elements:



```
cars.clear();
```

[Try it Yourself »](#)

LinkedHashSet Size

Use `size()` to count how many unique elements are in the set:

Example

```
cars.size();
```

[Try it Yourself »](#)

Note: Duplicate values are not counted - only unique elements are included in the size.

Loop Through a LinkedHashSet

Loop through the elements of a `LinkedHashSet` with a **for-each** loop:

Example

```
LinkedHashSet<String> cars = new LinkedHashSet<>();  
// Add elements...  
  
for (String car : cars) {
```



HashSet vs LinkedHashSet

Feature	HashSet	LinkedHashSet
Order	No guaranteed order	Insertion order preserved
Duplicates	Not allowed	Not allowed
Performance	Faster	Slightly slower (due to order tracking)

Tip: Use **HashSet** when you only care about uniqueness and speed. Use **LinkedHashSet** when order matters.

The var Keyword

From Java 10, you can use the **var** keyword to declare a **LinkedHashSet** variable without writing the type twice. The compiler figures out the type from the value you assign.

This makes code shorter, **but many developers still use the full type for clarity**. Since **var** is valid Java, you may see it in other code, so it's good to know that it exists:

Example

```
// Without var
LinkedHashSet<String> cars = new LinkedHashSet<String>();
```



The Set Interface

Note: Sometimes you will see both `Set` and `LinkedHashSet` in Java code, like this:

```
import java.util.Set;
import java.util.LinkedHashSet;

Set<String> cars = new LinkedHashSet<>();
```

Try it Yourself »

This means the variable (cars) is declared as a `Set` (the interface), but it stores a `LinkedHashSet` object (the actual set). Since `LinkedHashSet` implements the `Set` interface, this is possible.

It works the same way, but some developers prefer this style because it gives them more flexibility to change the type later.

[< Previous](#)[Sign in to track progress](#)[Next >](#)

[Tutorials ▼](#)[References ▼](#)[Exercises ▼](#)[Sign In](#)[CSS](#) [JAVASCRIPT](#) [SQL](#) [PYTHON](#) [JAVA](#) [PHP](#) [HOW TO](#) [W3.CSS](#) [C](#)

COLOR PICKER

[REMOVE ADS](#)[PLUS](#)[SPACES](#)

[Tutorials ▼](#)[References ▼](#)[Exercises ▼](#)[Sign In](#)[CSS](#)[JAVASCRIPT](#)[SQL](#)[PYTHON](#)[JAVA](#)[PHP](#)[HOW TO](#)[W3.CSS](#)[C](#)[FOR BUSINESS](#)[CONTACT US](#)

Top Tutorials

[HTML Tutorial](#)
[CSS Tutorial](#)
[JavaScript Tutorial](#)
[How To Tutorial](#)
[SQL Tutorial](#)
[Python Tutorial](#)
[W3.CSS Tutorial](#)
[Bootstrap Tutorial](#)
[PHP Tutorial](#)
[Java Tutorial](#)
[C++ Tutorial](#)
[jQuery Tutorial](#)

Top References

[HTML Reference](#)
[CSS Reference](#)
[JavaScript Reference](#)
[SQL Reference](#)
[Python Reference](#)
[W3.CSS Reference](#)
[Bootstrap Reference](#)
[PHP Reference](#)
[HTML Colors](#)
[Java Reference](#)
[AngularJS Reference](#)
[jQuery Reference](#)

Top Examples

[HTML Examples](#)
[CSS Examples](#)
[JavaScript Examples](#)
[How To Examples](#)
[SQL Examples](#)
[Python Examples](#)
[W3.CSS Examples](#)
[Bootstrap Examples](#)
[PHP Examples](#)
[Java Examples](#)
[XML Examples](#)
[jQuery Examples](#)

Get Certified

[HTML Certificate](#)
[CSS Certificate](#)
[JavaScript Certificate](#)
[Front End Certificate](#)
[SQL Certificate](#)
[Python Certificate](#)
[PHP Certificate](#)
[jQuery Certificate](#)
[Java Certificate](#)
[C++ Certificate](#)
[C# Certificate](#)
[XML Certificate](#)





Tutorials ▼

References ▼

Exercises ▼



Sign In



- CSS
- JAVASCRIPT
- SQL
- PYTHON
- JAVA
- PHP
- HOW TO
- W3.CSS
- C

correctness
of all content. While using W3Schools, you agree to have read and accepted our [terms of use](#), [cookie and privacy policy](#).

Copyright 1999-2025 by Refsnes Data. All Rights Reserved. W3Schools is Powered by W3.CSS.